VISUAL NOTES • COMPLETE GUIDE

Senior Engineers’ AI Coding Agent Playbook

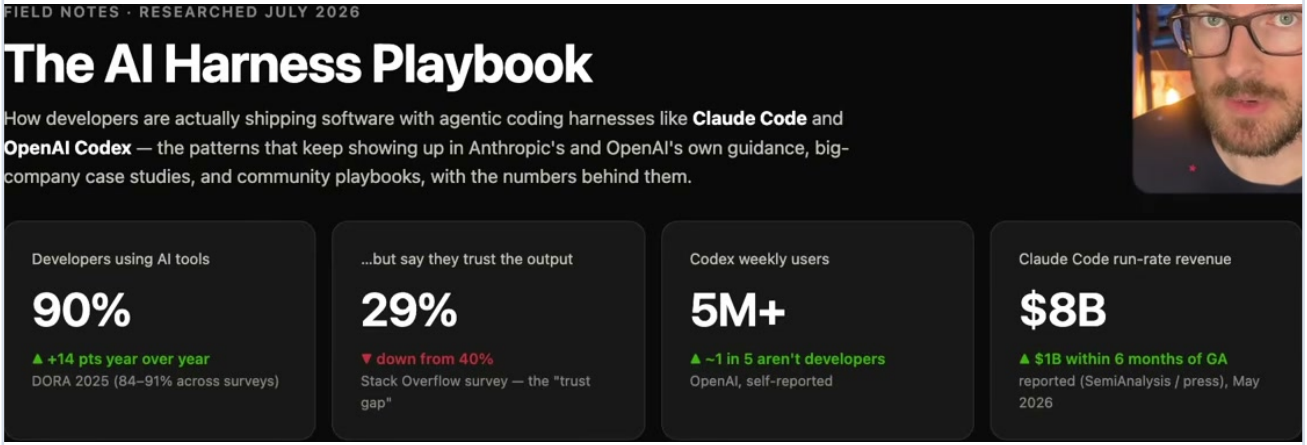
How Senior Engineers Use AI Coding Agents

29:19

- 13 substantive visual(s), selected by content
- Repeated builds, presenter-only shots, and filler excluded
- Output length expands or contracts with the source material
- Claims, uncertainty, and caveats are labeled clearly

https://www.youtube.com/watch?v=YdzMtb3jjfQ

Adoption is high while trust remains low



NOTES

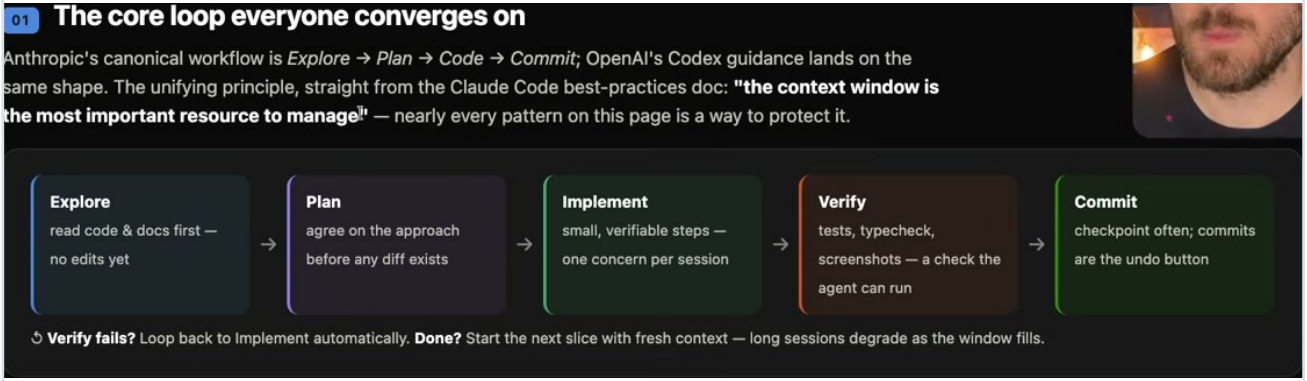
- The visual contrasts widespread AI-tool adoption with substantially lower reported trust in generated output.
- Codex usage is explicitly self-reported, while the Claude Code revenue figure comes through press reporting.
- The survey populations, definitions, and reporting dates differ, so the figures are not one controlled comparison.
- The operational response is to improve the engineering harness around the model rather than relying on prompts alone.

CORE IDEA

Reliable AI coding depends on the workflow, context, and checks surrounding the model.

Source: https://www.youtube.com/watch?v=YdzMtb3jjfQ

The core loop protects context and verification



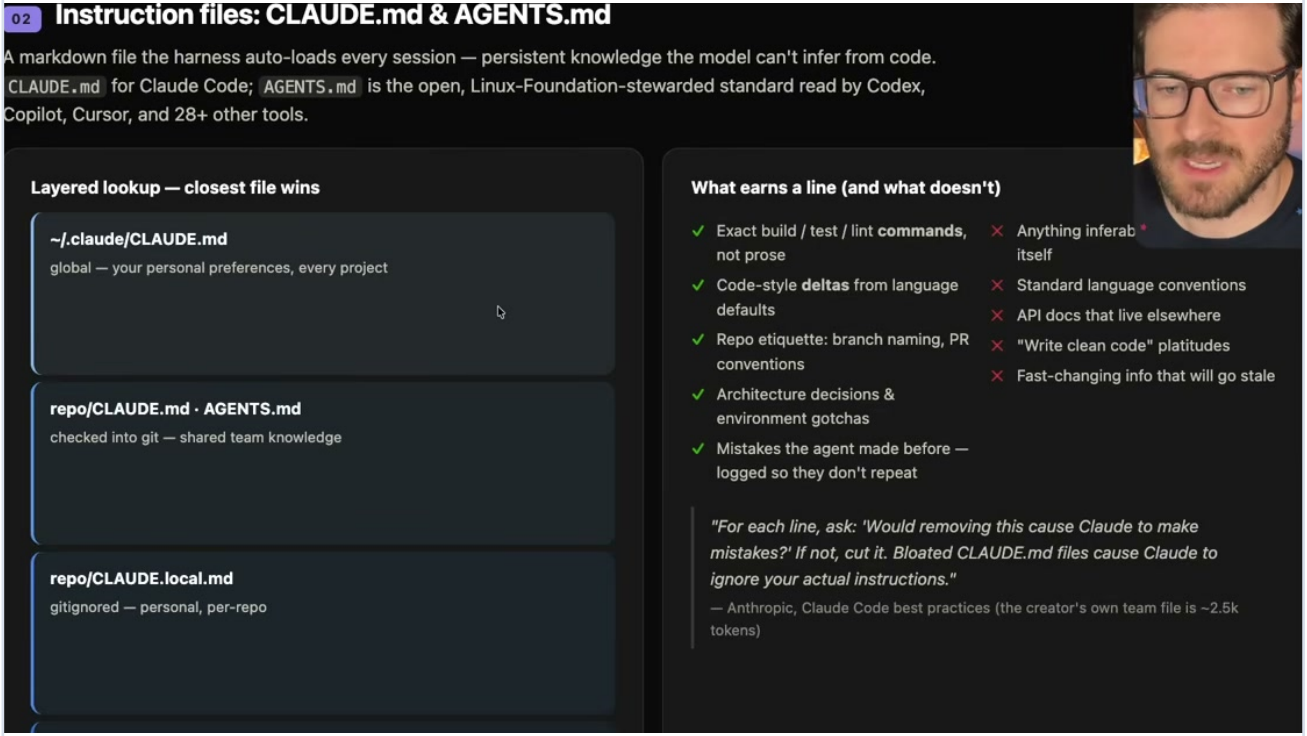
NOTES

- Explore relevant code and documentation before allowing edits.
- Make the plan reviewable before code changes, then implement one bounded concern at a time.
- Verification is a loop: failed tests, type checks, or screenshots return work to implementation.
- Frequent commits provide rollback points and durable history for debugging and future agents.
- Fresh contexts are recommended for new work slices because long sessions accumulate irrelevant state.

CORE IDEA

Treat agentic coding as a repeatable engineering loop, not a prompt followed by blind acceptance.

Instruction files should contain only high-impact constraints



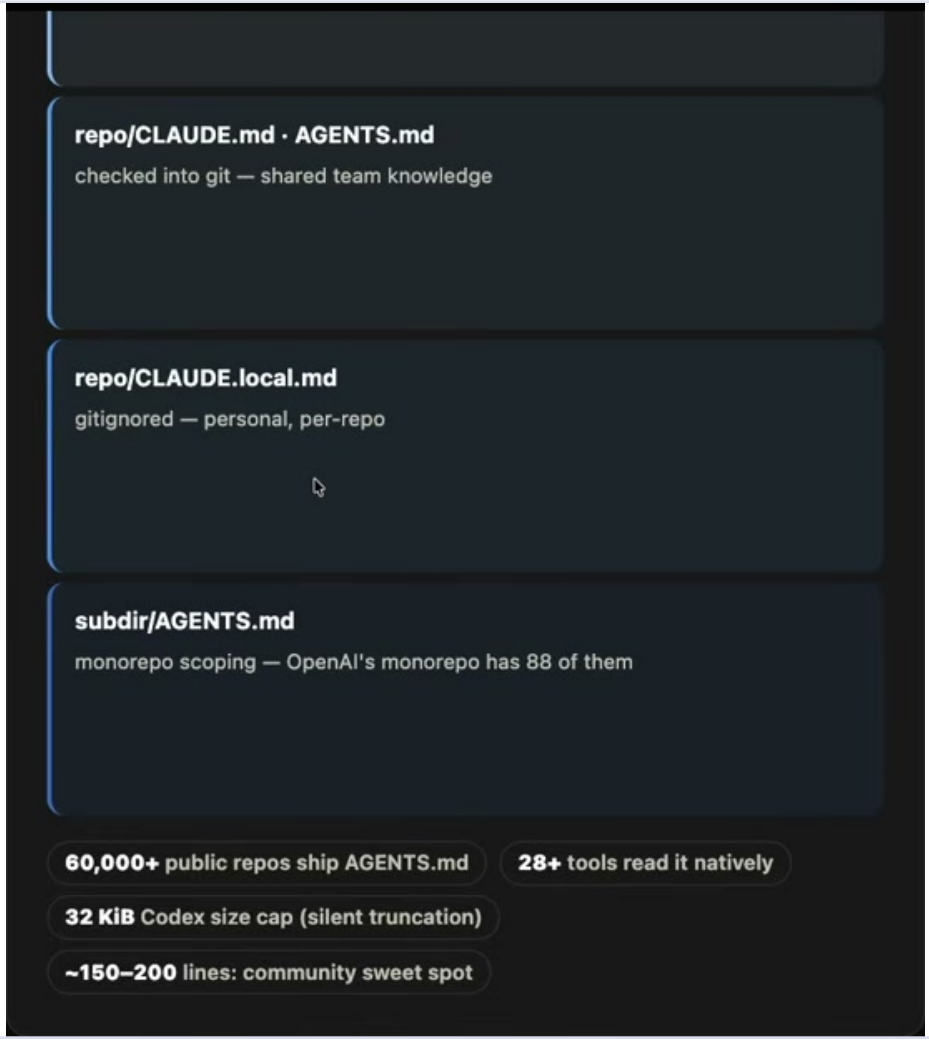
NOTES

- CLAUDE.md and AGENTS.md provide durable project knowledge that may not be obvious from code.
- Layered lookup lets repository or directory rules refine broader personal defaults.
- Useful lines include exact commands, local conventions, repository etiquette, caveats, and repeated mistakes.
- Generic advice, duplicated documentation, inferable facts, and rapidly changing values dilute useful instructions.
- Tool-count and adoption numbers are time-sensitive slide claims.

CORE IDEA

Instruction files work best as a small set of constraints that materially prevent mistakes.

Scope instructions locally and stay below tool limits



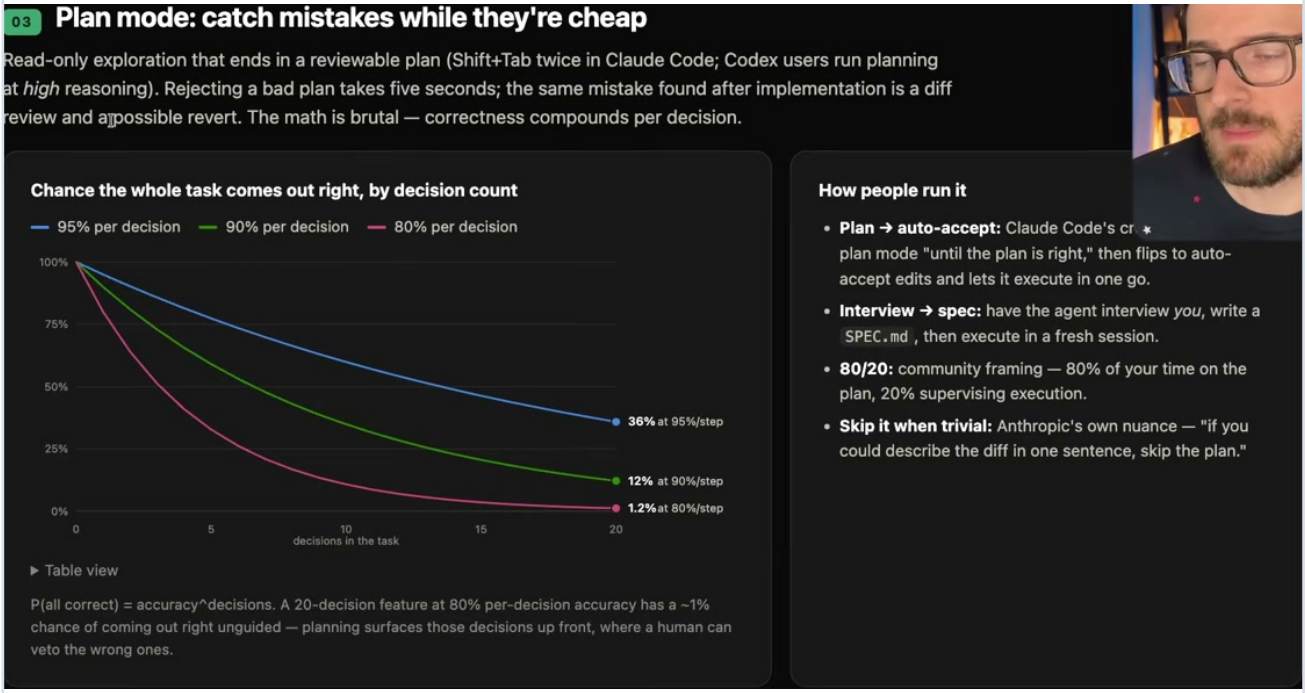
NOTES

- Repository files encode shared policy while local files preserve personal settings without committing them.
- Subdirectory instruction files narrow scope inside monorepos.
- The slide reports a 32 KiB Codex cap and warns that longer content may be silently truncated.
- Repository counts, tool counts, and the suggested 150–200-line range are community claims rather than universal standards.

CORE IDEA

Put rules close to the code they govern and keep them small enough to load reliably.

Plan important decisions before they become code



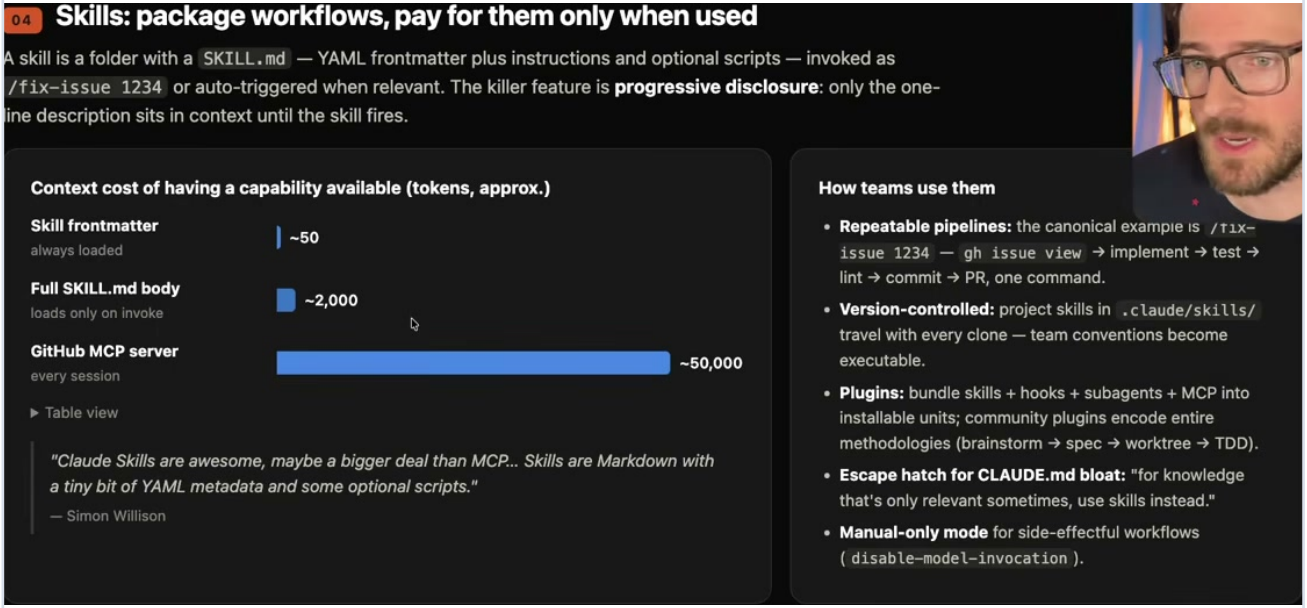
NOTES

- The chart illustrates how small per-decision error rates compound as a task accumulates decisions.
- Its endpoint values follow the simplifying formula accuracy raised to the number of decisions.
- Planning exposes architectural, product, performance, security, and UX choices before implementation.
- Agent interviews can reveal missing requirements and produce a reviewable SPEC.md.
- The 80/20 planning-versus-execution split is advice, not an experimentally established optimum.

CORE IDEA

Review costly decisions while they are still cheap to change.

Skills load specialized context only when needed



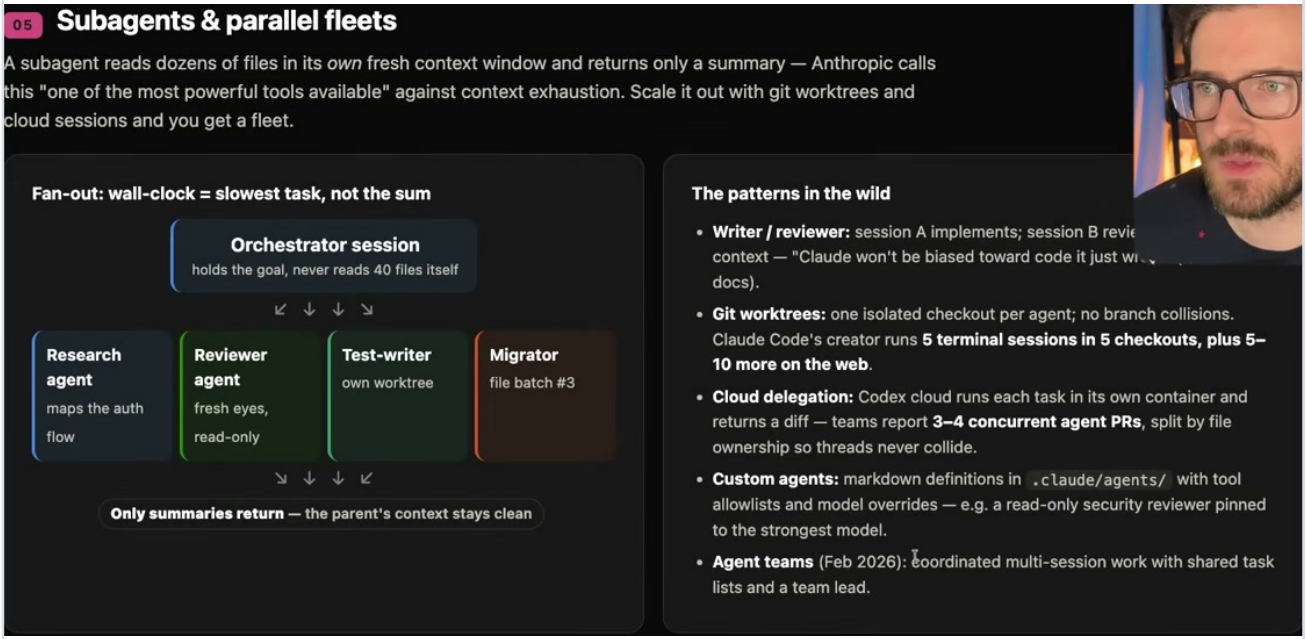
NOTES

- A skill packages SKILL.md instructions, references, tools, and optional scripts behind a trigger.
- Progressive disclosure keeps only a short description in routine context and loads the full body on demand.
- Teams can version repeatable issue-to-PR pipelines and shared procedures as skills.
- Occasional knowledge can move out of global instruction files, reducing context pressure.
- Displayed token counts are approximate illustrations, not universal measurements.

CORE IDEA

Package specialized workflows so their detailed instructions consume context only when relevant.

Subagents parallelize work without polluting parent context



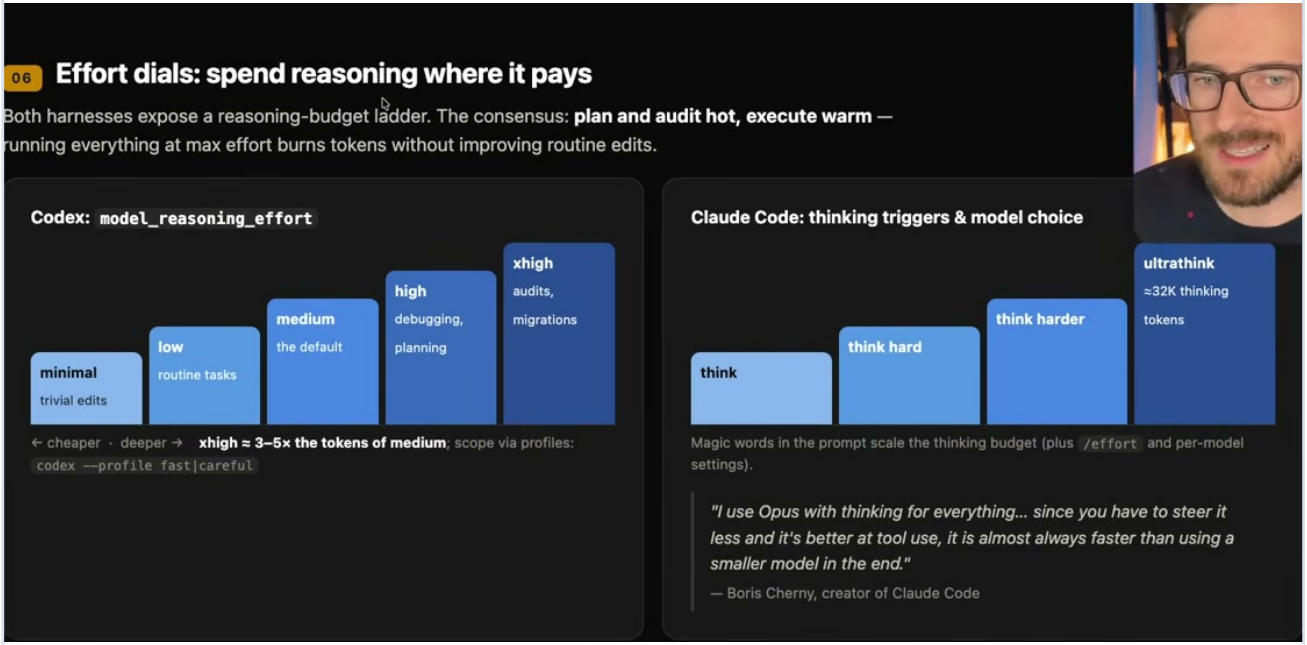
NOTES

- An orchestrator delegates research, review, testing, or migration to workers with separate context windows.
- Only summaries return, which prevents the parent context from absorbing every intermediate detail.
- Writer and reviewer separation supplies fresh context and reduces self-review bias.
- Git worktrees isolate checkouts but do not automatically prevent logical or merge conflicts.
- Parallelism saves time only when tasks are sufficiently independent and integration remains controlled.

CORE IDEA

Isolate parallel work and return compact results rather than every working detail.

Reasoning effort is a cost–quality control



NOTES

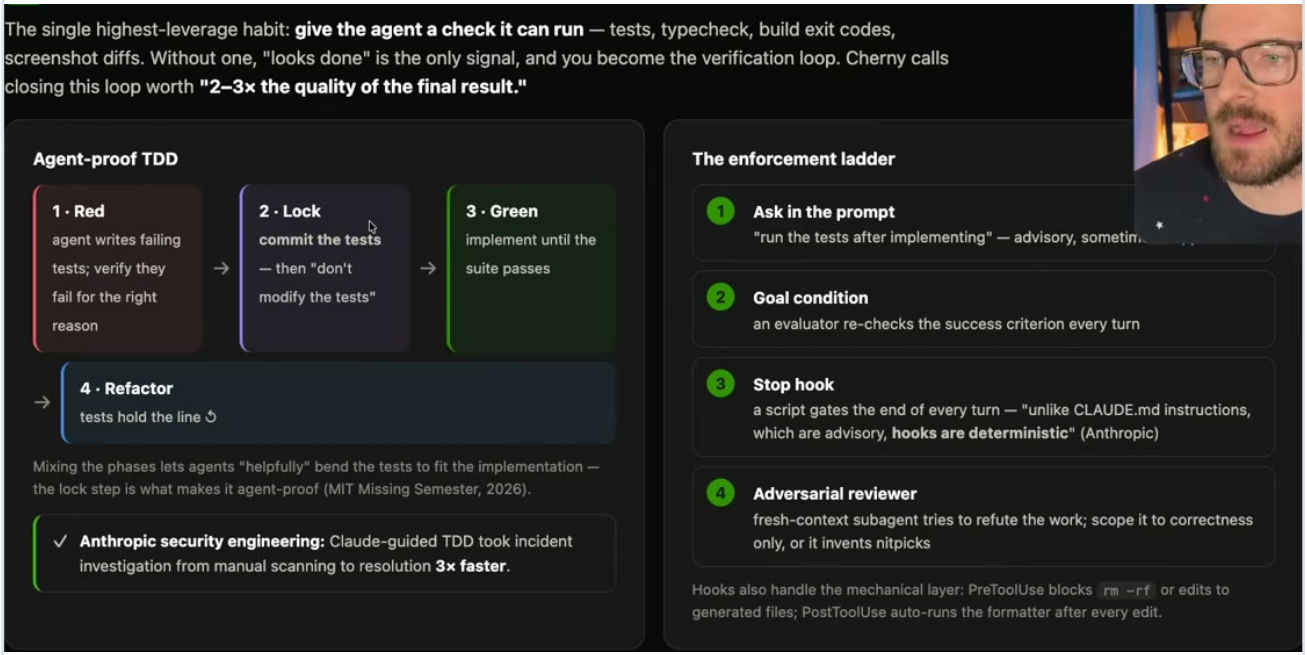
- Codex exposes effort levels from minimal through xhigh, while Claude Code uses thinking triggers and model choice.
- Higher effort is recommended for planning, debugging, audits, migrations, and ambiguous work.
- Routine edits may consume more tokens without receiving proportional quality gains at maximum effort.
- The displayed token multipliers and quality claims vary by model, prompt, and task and are not backed by benchmark methodology here.

CORE IDEA

Spend deep reasoning where errors are expensive rather than maximizing it for every task.

Source: https://www.youtube.com/watch?v=YdzMtb3jjfQ

Agent-proof TDD keeps the success criterion fixed



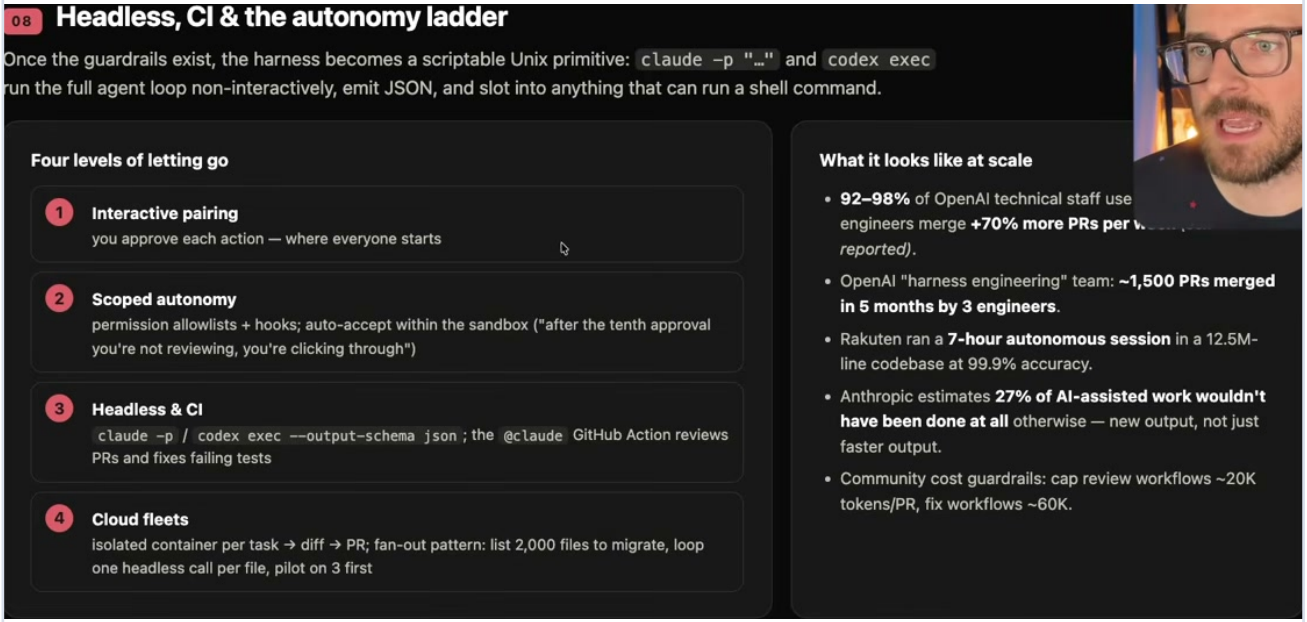
NOTES

- Write a failing test, confirm the intended failure, and commit the test before implementation.
- The agent implements until the locked suite passes, then refactors while tests hold the line.
- Prompts are advisory; goal checks, hooks, and executable gates provide stronger enforcement.
- A fresh-context reviewer adds adversarial inspection without sharing the implementation agent's assumptions.
- Passing tests still cannot prove requirements, security, or usability that the suite does not encode.

CORE IDEA

Define success with checks the implementation agent can run but cannot casually rewrite.

Autonomy should increase only after guardrails exist



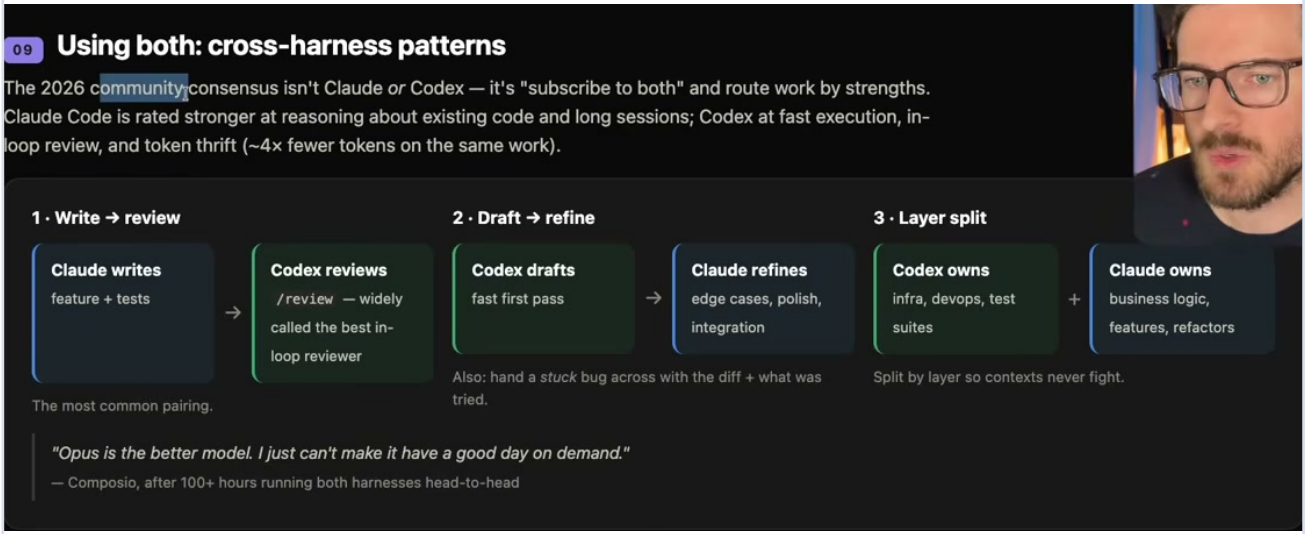
NOTES

- The autonomy ladder moves from interactive approval to scoped permissions, headless CI, and isolated cloud fleets.
- Headless commands can emit structured output and participate in pull-request workflows.
- Small pilots are recommended before fanning migrations across large file sets.
- Permissions, executable checks, sandboxes, and reviewable diffs must precede greater autonomy.
- Scale figures shown are internal reports or case studies and measure different outcomes.

CORE IDEA

Earn autonomy through deterministic controls, then scale with isolated and reviewable tasks.

Route work across harnesses by task strength



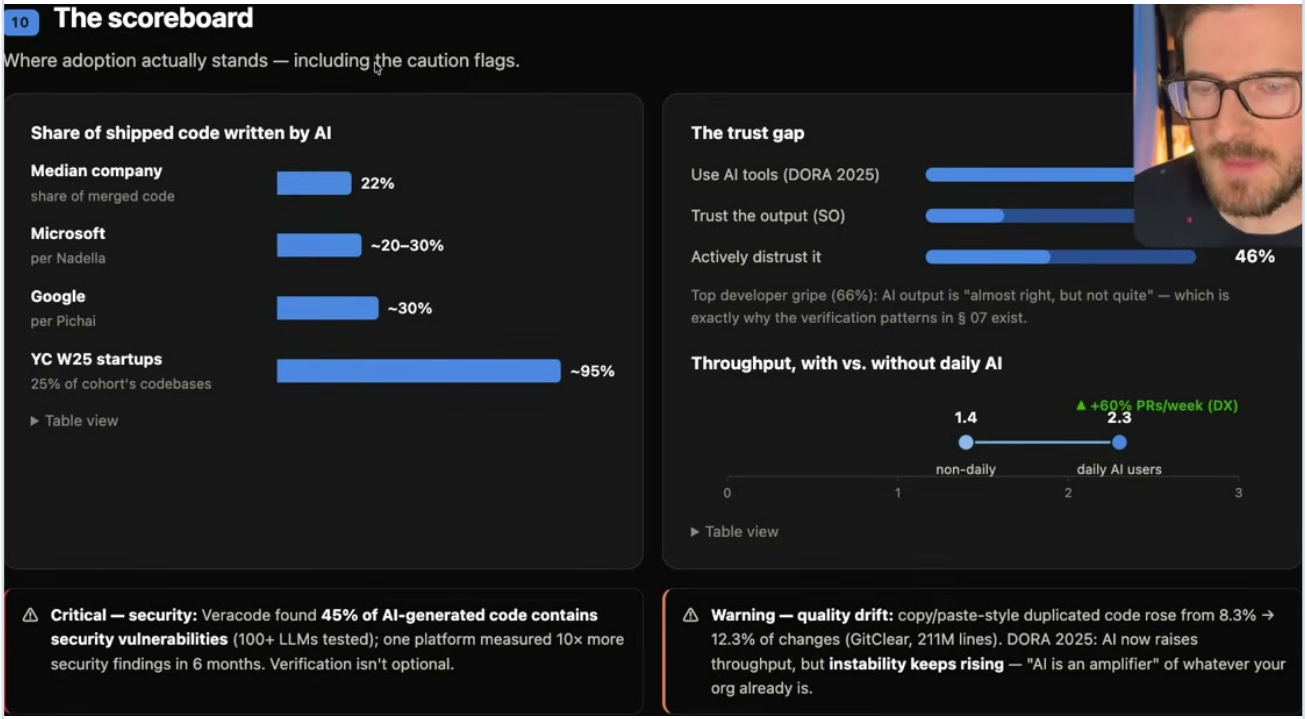
NOTES

- Writer/reviewer separates feature implementation from a fresh model's review.
- Draft/refine uses a fast first pass followed by edge-case, polish, and integration work.
- Layer split assigns infrastructure and tests separately from business logic and refactors.
- Passing a diff and prior attempts preserves useful context when transferring a stuck problem.
- Relative-strength and token-efficiency claims are community reports, not controlled benchmarks.

CORE IDEA

Use models as complementary workers instead of forcing one harness across every phase.

Throughput gains coexist with trust and quality risks



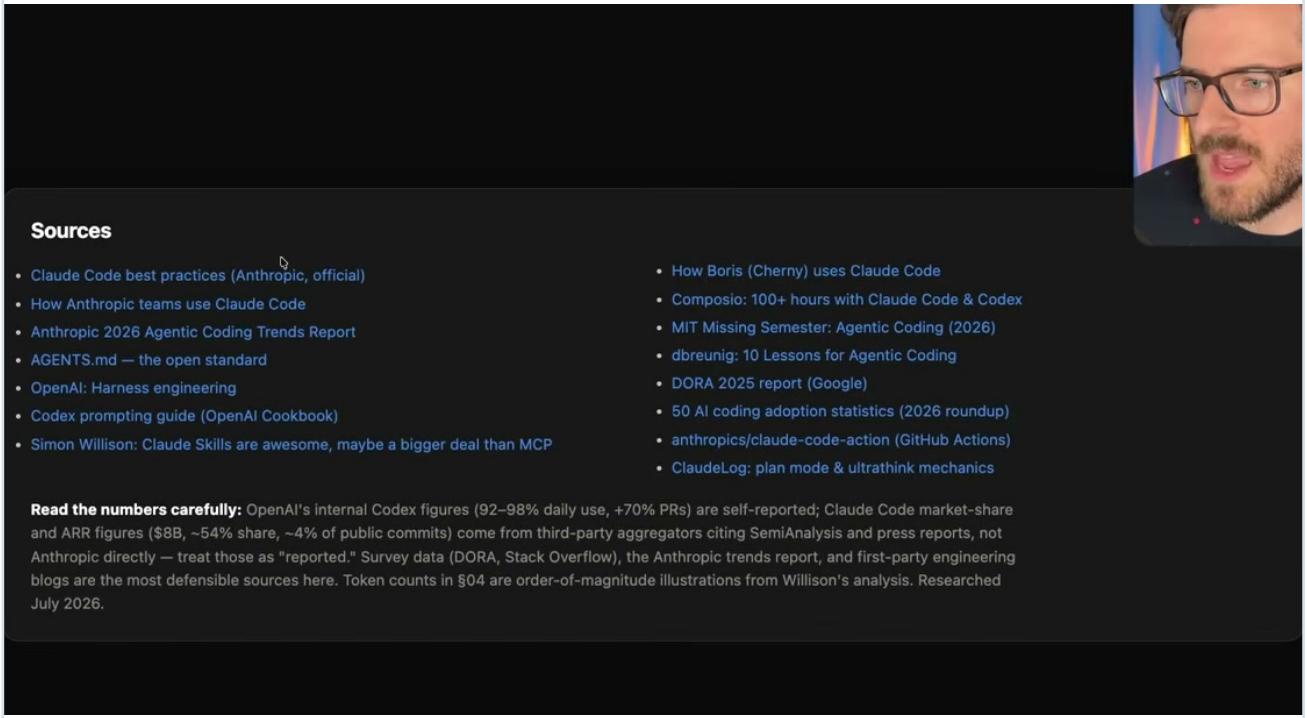
NOTES

- Reported AI-written-code shares vary widely because organizations use different definitions and evidence.
- Usage, trust, code share, and pull-request throughput measure separate phenomena.
- Higher throughput does not establish correctness, security, customer value, or maintainability.
- Security findings, duplicated code, and rising instability support the need for independent verification.
- The displayed figures come from external reports rather than demonstrations in the video.

CORE IDEA

AI amplifies delivery capacity and existing weaknesses at the same time.

Source quality varies across the headline numbers



NOTES

- The source list mixes first-party engineering guidance, surveys, internal claims, press reports, and third-party analysis.
- Similar-looking percentages may use different definitions, samples, and methodologies.
- OpenAI usage and pull-request figures are explicitly identified as self-reported.
- Skill-versus-MCP token numbers are described as order-of-magnitude illustrations.
- The research date is July 2026, so tool counts and adoption figures are time-sensitive.

CORE IDEA

Use the workflow patterns directionally while keeping self-reported metrics and approximations labeled.